A smaller complete clause calculus for iPRA

Raw terms, ordinary substitution, and two evaluation cases

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

This report gives a strict local simplification of the newest raw-clausal presentation in ConciseIPRA. Delete the direct root-composition case of its evaluation rule. Reflexivity and the remaining zero-evaluation case derive the deleted inference through an identity frame, while successor evaluation is unchanged. The resulting calculus has exactly the same sequents on exactly the same language. It is therefore equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic, with whole numerical domains, actual equality, and numerical composite isomorphism graph $x = y$.

The complete unchanged-interface checker decreases from 3046 to 2999 normalized Python bytes, from 853 to 848 significant tokens, and from 993 to 986 AST nodes. An explicit compiler preserves each conclusion and replaces k root steps in an N-node proof by an $(N + 2k)$-node proof. A supplemental local lower bound shows that the three-node replacement is optimal in the pure reduced arithmetic fragment.

The clause presentation and the identity-frame deletion are both already present, separately, in the incoming reports. This deliverable integrates them and verifies the exact matched saving; it does not claim a new foundational breakthrough, an absolute minimum, or a repository-wide checker-size record. The complete axiomatization, interpretation argument, inherited arithmetic proofs, executable verifier, and proof compiler are supplied.

Contents

1 The result and its comparison boundary 3

2 The complete proposed axiomatization 3

3 The deleted rule and its exact replacement 5

4 Recovering computation and ordinary equality 7

5 Logical completeness and the full iPRA interpretation 8

6 The complete checker and executed validation 10

7 Other simplifications tested and exact limits 12

8 Further research questions 13

Inherited logical and arithmetic reference proofs 14

A Complementary introductions with one local assumption 14

B Arithmetic strength and both ordinary interpretation composites 20

C Internal scalar predecessor bootstrap 21

D From literal-equation induction to full intuitionistic PRA 26

E Pairing, primitive recursion, and internal numerical composites 31

F The complete recommended checker 34

1 The result and its comparison boundary

The source audit is pinned to ProveIt commit

`831081033253cca2115196cf86bb99c77ed2aeae`.

The inspected project is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` [1]. Its README and all five incoming archives were examined. The archive bytes were checked against their Git blob hashes at that snapshot. This matters because the incoming material is stronger than the eight-rule kernel summarized first in the README.

The immediate control is the newest raw-clausal checker `code/clausal_eval.py` inside `ipra_raw_clausal_2026-11-11.zip` [2]. It uses five proof instruction families: reflexivity, Eval, clause use, paired elimination, and paired introduction. Eval itself has three cases. Call this calculus $\mathcal{C}_3$.

The proposed calculus $\mathcal{C}_2$ keeps the same numerical terms, formulas, contexts, binding discipline, and proof format, and deletes one of those three cases. It combines the raw clause interface with the identity-frame argument already supplied in `ipra_equality_from_iteration_2026-11-11.zip` [3]. The result is a smaller integrated presentation, not a claim that either component theorem is new.

Theorem 1.1 (Exact local simplification)**.** *For every finite clause context* Γ *and every well-scoped raw formula* F*,*

$$\Gamma \vdash_{\mathcal{C}_2} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_3} F.$$

Both translations leave every end sequent—its terms, formula, and context clauses—literally unchanged. Consequently the ordinary realization of $\mathcal{C}_2$ *is bi-interpretable and equiconsistent with full first-order* iPRA*.*

There are still five serialized instruction families. If the two evaluation branches are counted as separate mathematical schemas, the displayed inventory has six schemas rather than the control's seven. The introduction schema retains its explicit mode test. No encoding-independent minimum follows from either counting convention.

Complete program	Bytes	Tokens	AST nodes
Exact raw-clausal control	3046	853	993
Reduced raw-clausal checker	**2999**	**848**	**986**
Reduction	47	5	7

These are whole-module measurements under the repository's CPython 3.12.14 procedure, including parsing, validation, scopes, substitution, inference, and process output. The source change is a deletion of one Eval case; the public contract is identical.

2 The complete proposed axiomatization

2.1 Terms, formulas, clauses, and ordinary readings

There is one numerical sort. Terms are raw finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

The iterator binds u only in its body h, not in its counter n or initializer a. The body may have passive numerical parameters and may contain further iterators. Every substitution in the main calculus is ordinary capture-avoiding substitution. In particular, $I(0, a; u.h)$ is not definitionally the same term as a.

The formula language is

$$F ::= (s = t) \mid D_{x,y}F, \qquad \bot_* := (0 = S0).$$

The distinct coordinates x, y are bound. A context is a finite reusable list of clause occurrences

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The clause binds y only in its consequent. Clauses are context objects, not additional formula constructors. Their ordinary intuitionistic readings are

$$(D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$
$$[A; y.B]^\circ = A^\circ \to \forall y B^\circ.$$

Equality has its actual numerical meaning. These readings define an interpretation; they do not add primitive inference rules.

Put $H = (0 = 0)$. An ordinary assumption F is an abbreviation for the constant clause

$$\mathrm{ord}(F) = [H; v.F], \qquad v \notin \mathrm{FV}(F).$$

Thus a local context written Γ, F below means $\Gamma, \mathrm{ord}(F)$, and its use is derived by clause use from H.

2.2 Every primitive rule

The two arithmetic evaluation cases are

$$\frac{\Gamma \vdash a = t \qquad \Gamma \vdash I(0, a; u.h) = r}{\Gamma \vdash r = t}\ \mathsf{V0}, \tag{V0}$$

$$\frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]}\ \mathsf{VS}. \tag{VS}$$

In particular, the major premise of VS is about the entire predecessor iterate. It is not an initializer equality. These two cases are serialized by the single instruction tag Eval.

The other four instruction families are exactly

$$\frac{}{\Gamma \vdash s = s}\ \mathsf{Refl}; \tag{R}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{U}, \qquad [A; y.B(y)] \in \Gamma; \tag{U}$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, [B(b, 0); y.B(b, y)] \vdash F}{\Gamma \vdash F}\ \mathsf{E}; \tag{E}$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \mathsf{I}, \tag{I}$$

where

$$\eta_{B,t}(z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

The symbol $\equiv$ means literal identity of syntax. An equation under a binder is not a literal equation. All instances are well scoped, and witnesses and clause targets may be arbitrary terms.

In E the parameter b is fresh for Γ, F and all passive parameters of B. In I the parameter z is fresh for Γ, t and all passive parameters of B. The displayed local assumptions are discharged. Context entries remain reusable inside each branch.

This is the entire primitive inventory. Computation equations, symmetry, transitivity, congruence, successor injectivity, numerical coverage, ordinary logical connectives and their rules, and arithmetic induction are derived. There is no first-class Q formula or separate hypothesis instruction.

2.3 Elementary structural facts

Refl proves H in every context. U applied to $[H; v.F]$ then proves the ordinary assumption F. Weakening and injective fresh renaming follow by induction on derivations. They leave the mode of I unchanged.

Ordinary-assumption cut is also admissible: to remove an occurrence $[H; v.F]$ supplied with a proof of F, replace each use of that clause by the supplied proof, weakened into the local context. Freshen eigenvariables first. This is induction on finite proof syntax. Arbitrary numerical substitution into proofs is not presumed here: it may change a nonliteral witness into literal zero and therefore change the primitive introduction mode.

3 The deleted rule and its exact replacement

The control additionally permits

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash r = t}\ \mathsf{Root}. \tag{Root}$$

The following inherited identity-frame argument removes it without using any equality laws in advance [3].

Lemma 3.1 (Identity-frame derivation)**.** *Let $W(s) = I(0, s; u.0)$. From $p : s = t$ and $q : s = r$ one derives $r = t$ by*

$$\boxed{\mathsf{Root}(p, q) := \mathsf{V0}(p, \mathsf{V0}(q, \mathsf{Refl}(W(s))))}.$$

Both supplied premise proofs occur once.

Proof. Refl gives $W(s) = W(s)$. Applying V0 with major premise $q : s = r$ gives $W(s) = r$. Applying V0 again, now with major premise $p : s = t$, gives $r = t$. Every step has the same retained clause context. The term $W(s)$ is well scoped whenever s is. No computation equation, symmetry, induction, logical rule, or proof-substitution theorem has been invoked. □

For example, the abstract root step $x = y,\ x = z \vdash z = y$ expands as

1. $W(x) = W(x)$, Refl;
2. $W(x) = z$, V0 from $x = z$ and line 1;
3. $z = y$, V0 from $x = y$ and line 2.

Theorem 3.2 (Literal preservation of every sequent)**.** *The identity map on language and contexts gives the equivalence* $\Gamma \vdash_{\mathcal{C}_3} F \iff \Gamma \vdash_{\mathcal{C}_2} F$.

Proof. Induct on a $\mathcal{C}_3$ proof tree. Replace each Root node by the identity-frame derivation and copy every other rule, translating its subproofs recursively. At E and I the same local clause and the same scope are introduced, so the induction hypothesis applies under exactly the required context. The reverse translation is inclusion: every rule of $\mathcal{C}_2$ is a rule of $\mathcal{C}_3$. □

The three control evaluation cases are disjoint on raw finite syntax. For zero evaluation, $I(0, a; u.h)$ has a as a proper subterm, and therefore cannot equal a literally. For successor evaluation, $I(Sc, a; u.h)$ has one more successor node than the required source $I(c, a; u.h)$, and therefore cannot literally equal it. Their counters also distinguish the two computation cases. Thus the compiler can identify Root by its literal matching condition without changing the classification of a retained computation step.

Corollary 3.3 (Exact compiler overhead)**.** *If a control proof has N inference nodes and k Root nodes, the displayed tree compiler produces exactly $N + 2k$ nodes.*

Proof. Each removed node becomes two V0 nodes and one Refl node. Its two premise subtrees occur once, and all other nodes are copied. □

This is a count of inference nodes, not certificate bytes. The new Refl label contains $W(s)$, so the byte overhead depends on the size of the original source term. The standalone verifier does not run this compiler or trust it; it checks the resulting certificate.

3.1 A local optimality bound for the replacement

The following additional observation bounds the arithmetic gadget; it does not bound the number of axioms or the size of a verifier.

Proposition 3.4 (Three new nodes are necessary)**.** *Take distinct free variables x, y, z and treat supplied proofs $p : x = y$ and $q : x = z$ as the only zero-cost opaque leaves. In the pure raw fragment with Refl, V0, and VS, every proof tree of $z = y$ contains at least two Eval nodes and one Refl node in addition to the supplied leaves. The identity-frame construction attains this bound.*

Proof. The last inference cannot be p, q, or Refl: their conclusions are respectively $x = y$, $x = z$, and a syntactically reflexive equation. It is therefore an Eval. Its minor premise has an iterator-headed left side and right side z, because Eval carries the minor's right side to the conclusion's left side. That minor cannot be p or q, whose left side is x; and it cannot be Refl, since a raw iterator term is not the variable z. Hence the minor itself ends in a second Eval.

Follow minor-premise edges from that Eval. The finite tree must eventually reach a leaf with iterator-headed left side. Neither opaque supplied leaf has that shape, so this leaf is a Refl instance. This proves the lower bound. The three-node expansion above supplies the matching upper bound. □

The restriction to this fragment is essential. Preprovided derived lemmas, additional inference rules, primitive cut, different term quotients, and a different certificate representation are outside the statement. It is not a global minimality claim.

4 Recovering computation and ordinary equality

All constructions in this section work under an arbitrary retained clause context. Here Root always denotes the derived three-node construction, never an added primitive.

For $p : s = t$ define

$$\mathrm{Sym}(p) = \mathsf{Root}(\mathsf{Refl}(s), p) : t = s.$$

For $p : s = t$ and $q : t = v$ define

$$\mathrm{Trans}(p, q) = \mathsf{Root}(q, \mathrm{Sym}(p)) : s = v.$$

Both iterator computation equations have direct proofs:

$$\begin{aligned} I(0, a; u.h) &= a \quad \text{by } \mathsf{V0}(\mathsf{Refl}(a), \mathsf{Refl}(I(0, a; u.h))), \\ I(Sc, a; u.h) &= h[I(c, a; u.h)/u] \quad \text{by } \mathsf{VS}(\mathsf{Refl}(I(c, a; u.h)), \mathsf{Refl}(I(Sc, a; u.h))). \end{aligned}$$

Lemma 4.1 (Congruence without root composition)**.** *For every fixed finite term template $h(u)$, a proof of $s = t$ gives a proof of $h(s) = h(t)$ using only Refl, V0, and VS.*

Proof. Set $A = I(0, s; u.h)$ and $B = I(S0, s; u.h)$, and let $p : s = t$. Construct

$$\begin{aligned} z_s &= \mathsf{V0}(\mathsf{Refl}(s), \mathsf{Refl}(A)) && : A = s, \\ c_s &= \mathsf{VS}(z_s, \mathsf{Refl}(B)) && : B = h(s), \\ z_t &= \mathsf{V0}(p, \mathsf{Refl}(A)) && : A = t, \\ \mathsf{VS}(z_t, c_s) & && : h(s) = h(t). \end{aligned}$$

The last minor has left side $B = I(S0, s; u.h)$ and the required predecessor iterate is literally A. This uses no earlier congruence or transitivity principle. Capture-avoiding body substitution handles all occurrences of u, including nested ones. □

Given a second premise $h(s) = r$, derived Root applied to it and the congruence equation yields the crossed one-template rule

$$s = t,\ h(s) = r \quad \Longrightarrow \quad r = h(t).$$

Applying that rule twice gives transport in an arbitrary literal equation template:

$$s = t,\ a(s) = b(s) \quad \Longrightarrow \quad a(t) = b(t).$$

The template is needed only in this derived construction, as an iterator body. Primitive certificates have no equality-template argument.

5 Logical completeness and the full iPRA interpretation

5.1 The missing case rule is derived

The introduction in Section 2 uses progress only for an equation matrix at literal witness zero. All numerical case introductions are nevertheless available. The only missing primitive case has matrix $E(x, y)$ an equation and witness 0. Put $c = I(0, 0; u.0)$. Its syntax is not zero, and computation proves $c = 0$.

Suppose the desired case premise is $\Gamma, E(0, 0) \vdash E(0, Sz)$. Under $E(c, 0)$, equation transport gives $E(0, 0)$; ordinary-assumption cut inserts the given premise; transport back gives $E(c, Sz)$. Primitive I at witness c uses its case mode and concludes the desired $D_{x,y}E(x, y)$. No compound-formula transport is needed here.

Transport through a compound D formula then follows by external structural induction on the formula: eliminate the source D to obtain its clause, introduce the target D by the just-derived case rule, translate the base by the induction hypothesis, use the clause at Sz, and translate its conclusion back. Only a strictly smaller formula is used in the inductive transport.

5.2 The omitted closure binder is definable

For a fresh unused coordinate w, write

$$q_y B(y) := D_{w,y} B(y).$$

It has the following derived interfaces:

- From $\Gamma, B(0) \vdash B(Sz)$ derive $q_y B$ by case introduction with unused witness coordinate.
- From $q_y B$ and $B(0)$ derive $B(t)$ by E and U.
- From a retained clause $[B(0); y.B(y)]$ derive $q_y B$ by case introduction, using that clause at Sz.

The last property is the eta expansion of the opened clause. For a literal equation $E(y)$, I with unused witness 0 derives $q_y E$ from $\Gamma, E(z) \vdash E(Sz)$. With the base equation and closure elimination this is contextual equation induction.

These constructions give the explicit comparison to the repository's native Eq/Q/D calculus: leave terms and equations unchanged, translate D recursively, and translate $Q_y B$ as $q_y B$. Ordinary assumptions become constant clauses. In a translated native elimination branch, the opened clause eta-expands to the translated Q assumption, and ordinary-assumption cut inserts the translated branch proof. Conversely a clause created by E is represented by a native Q assumption, while a constant clause is represented by its ordinary formula assumption. U becomes respectively native QE or ordinary assumption use. The remaining rules are computation, equality, and the same introductions or their derived forms.

This clause comparison is inherited from the two clausal reports [4, 2]. The exact same-sequent theorem of Section 3 makes it applicable without modifying any interpretation map. On Eq/D formulas the inclusion followed by translation is literally the identity. In the other direction, the native equivalence

$$Q_y B(y) \dashv\vdash D_{w,y} B(y)$$

gives the formula round trip by structural induction.

5.3 The ordinary arithmetic theory

Let T_I be intuitionistic first-order logic with actual equality and genuine falsity. Each fixed finite iterator body is elaborated into an ordinary function symbol. If $h(u, \vec{p})$ is an earlier term, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

The other axioms are coverage and separation,

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \longrightarrow \bot,$$

and contextual induction for every literal equation. There are no function variables, and no total universal evaluator for codes of primitive recursive programs.

The inherited native interpretation derives ordinary intuitionistic logic, numerical coverage, and explosion, and proves exact equivalence with T_I. The detailed logical and arithmetic reference proofs are reproduced in the appendices. In particular, the ordinary theory is not just the iterator equations interpreted in a singleton algebra: the falsity bridge and numerical cases are part of the established native interpretation.

Every primitive rule of $\mathcal{C}_2$ is sound in T_I under the readings in Section 2. V0 and VS follow from the computation equations and equality. U and E are ordinary intuitionistic eliminations. The case mode of I uses zero/successor coverage; the progress mode uses induction for the literal equation. The reverse interpretation follows from the recovered native calculus and its ordinary logical macros.

5.4 From scalar arithmetic to standard iPRA

Here iPRA is full intuitionistic first-order logic with equality, one function symbol for every finite primitive recursive definition, its defining equations, successor injectivity and separation, and induction for quantifier-free formulas with parameters. This is the repository's target theory, not only its equational fragment.

The inherited scalar bridge proceeds in a noncircular order:

1. Construct a scalar predecessor P and prove $P0 = 0$ and $P(Sn) = n$ using literal-equation induction and numerical cases.
2. Construct truncated subtraction and a faithful Boolean characteristic of actual equality at arbitrary domain elements.
3. Reflect each fixed quantifier-free formula into an equation and obtain its induction instance.
4. Construct pairing and projections, then compile each fixed primitive recursion into scalar iteration.

These are inherited mathematical results [8, 9, 10]; the corresponding source proofs are included unchanged.

For clarity, let π, L, R be the derived pairing and projections. An ordinary recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is represented by the finite scalar term

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), R(u), \vec{p}))\big),$$
$$f^*(n,\vec{p}) = R(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection equations then prove both defining equations for f^*. Pairing is used only after its scalar construction is available.

5.5 Both composites and equiconsistency

Interpret each fixed scalar iterator in iPRA by ordinary primitive recursion with an update that ignores its counter. Interpret each primitive recursive definition in the other direction by the scalar compiler. Both domain formulas are $x = x$, and equality is actual equality.

Equation induction proves uniqueness for each fixed iterator and for each fixed general primitive recursion. External induction on a finite description therefore proves, internally in the appropriate theory, that each double-translated numerical operation equals the original operation. Thus the numerical composite isomorphism has graph $x = y$, on the whole domain, including nonstandard elements.

The native logical comparison supplies both formula round trips. For D the key equivalence is its ordinary reading

$$D_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big),$$

where the starred connectives are the derived native macros. The forward proof opens the clause and specializes it under the base assumption; the backward proof uses the supplied universal in case introduction. Structural induction handles all formulas.

Finally the identity translation $\mathcal{C}_2 \leftrightarrow \mathcal{C}_3$ changes neither composite. It also preserves the designated contradiction $0 = S0$. Translating derivations of contradiction in both directions proves equiconsistency. This proves the requested bi-interpretability, rather than merely agreement of term values on standard numerals.

6 The complete checker and executed validation

6.1 Public certificate contract

The top-level JSON object contains exactly `goal` and `proof`. Its goal is a closed Eq/D formula. The mathematical calculus supports open sequents; the public executable checks closed theorems. Both compared programs use this same contract.

Zero is the integer 0. Negative integers name free parameters introduced by proof branches. Positive integers are local bound indices starting at 1. Successor is `[0,t]`, iteration is `[n,a,h]`, equality is `[s,t]`, and D is `[B]`. The expected sort distinguishes a successor term from an equality formula. Only an iterator body gains one bound coordinate; a D body gains two. Every proof-supplied term and formula is validated in its actual scope.

Tag	Certificate form	Operation
`r`	`["r",term]`	Reflexivity
`v`	`["v",major,minor]`	V0 or VS
`u`	`["u",index,target,premise]`	Clause use
`e`	`["e",goal,major,branch]`	Paired elimination
`i`	`["i",goal,witness,branch]`	Paired introduction

The opening operation inserts only terms scoped at the current free-parameter level; their free coordinates are negative. Consequently insertion beneath a binder requires no renumbering of these free coordinates. The opening traversal still increments the protected bound depth and removes the opened bound index. The freshness discipline is enforced by assigning one new negative level per E or I branch and checking its conclusion back at the old scope.

A structural invariant of successful derivations is that every returned result is a well-scoped formula. Refl establishes it; U uses a well-scoped retained clause; E and I validate their conclusions and preserve branch scope; Eval checks matching equation and iterator shapes. A nonsuccessor numerical counter cannot pass the VS match: if it is a list of length two, the term invariant already forces its head to be 0.

The executable rejects malformed input, failed matching, invalid scopes, and mismatched goals. It uses explicit failure rather than `assert`, so Python's optimization flag does not remove certificate checks. As with any finite interpreter, resource limits may reject very large valid proof trees; no bounded runtime can accept all finite certificates of unbounded size.

6.2 Measurement procedure and exact diff

The primary measure parses the complete module, removes its module docstring (neither compared checker contains function docstrings), applies `ast.unparse`, appends one line feed, and counts UTF-8 bytes, significant Python tokens, and AST nodes under CPython 3.12.14. Gzip-9 size is reported separately as information: it falls from 990 to 979 bytes.

The source change removes the control's test `if b != s:` and makes its iterator matching unconditional. All surrounding validation and the four other instruction families are byte-for-byte the same. The archive includes the exact normalized diff, both complete sources, and a script that recomputes all measurements. This controlled deletion is the basis of the simplicity claim; it does not depend on renaming variables, changing the certificate encoding, or omitting validators.

6.3 Verification performed

The executed main-suite evidence records:

- 32 inherited valid certificates and 5 targeted additional certificates, translated and checked in both directions;
- 24 inherited invalid certificates, all rejected;
- four original certificates that exercise the deleted branch: accepted by the control, rejected by the reduced checker before translation, and accepted after translation;

- 260 command-line executions of the proposed source and its normalized source, under ordinary Python and Python `-O`;
- 112 additional command-line runs from the unchanged inherited test suite;
- identical end formulas, no surviving Root nodes, compiler idempotence, and the exact $N + 2k$ node count.

The cases exercise actual use of an induction hypothesis, nested binders, capture-avoiding witnesses, closure specialization, zero-source equality, nested numerical templates, malformed values and scopes, wrong antecedents, and an iterator-counter attack. These tests check implementations and specific proof transformations. They are not a proof-assistant verification of every certificate or of the bi-interpretation theorem.

7 Other simplifications tested and exact limits

7.1 The zero-normal single-evaluation alternative

A companion combines the same clause interface with the existing zero-normal single-Eval arithmetic [3, 6]. Contract $I(0, a; u.h)$ to a during substitution and write the smart iterator as $\mathcal{I}(c, a; u.h)$. Its only evaluation schema is

$$\frac{\mathcal{I}(c, a; u.h) = t \qquad I(Sc, a; u.h) = r}{r = h\langle t/u\rangle},$$

where angle brackets denote normalized substitution. Refl plus this rule recovers equality and computation by the one-step identity frame $I(S0, s; u.u)$. The same three clause operations complete the five-family presentation.

This is a cleaner arithmetic interface, but canonical syntax and normalization must be counted. The companion's complete checker measures 3050 bytes, 871 tokens, and 1011 AST nodes. A matched canonical control retaining Root measures 3107, 879, and 1023. Thus Root deletion improves that interface too, but the 2999-byte raw presentation remains smaller under the chosen primary measure.

Its proof translation must repair an introduction whose nonliteral witness normalizes to zero. The raw alias $I(0, 0; u.0)$ disappears. The surviving canonical alias $I(S0, 0; u.0)$, together with equation transport and ordinary-assumption cut, supplies the required case-mode proof. The companion includes that translation and a mode-sensitive test; it is not silently treating primitive I as stable under arbitrary normalization.

7.2 The smaller compact codecs are not surpassed

The repository's 516-byte profile uses a different guarded language and a backwards-consumed instruction stream [12]. It already combines source-selected equality and closure specialization and does not contain the three-case Eval interface being reduced here. It does not validate the complete integer/list input tree or enforce its public-goal grammar. Its paired strict version validates that tree and the closed guarded public-goal grammar, and measures 1376 bytes. Even that strict version remains smaller than this report's unrestricted raw Eq/D checker.

There is also a closer comparison: the incoming identity-frame report's Eq/Q/D checker has 2996 normalized bytes, 954 tokens, and 1019 AST nodes [3]. The new clause checker is three bytes

larger, although it uses 106 fewer tokens and 33 fewer AST nodes. This is another reason to claim a matched improvement of the raw clause presentation, rather than a minimum even over every transparent checker.

The exact source files and a contract audit were checked. These differences explain the comparison boundary; they do not turn the matched 3046-to-2999 saving into a global record. The relevant conclusion is strictly local: a complete program for the same full raw public language and certificate format has become smaller in all three primary coordinates.

7.3 Why no absolute simplest claim is made

Rule counts depend on how syntax, normalization, schemas, and proof instructions are divided. Source size additionally depends on the implementation language and public contract. This report proves an explicit deletion and measures its unchanged-interface consequence. It neither proves a lower bound over all bi-interpretable theories nor establishes that the current repository-wide source-size frontier has been beaten. The original request for an absolute simplest theory therefore remains open.

8 Further research questions

1. **Formalize the whole clause interface.** The repository formalizes the native arithmetic bridge and the three-case Eval calculus. A useful next target is a Lean development of clause contexts, their scope operations, the two-case calculus, and both complete interpretation composites. The written proof here is short enough that this is a concrete verification project.
2. **Translate the reduction into the compact protocol.** The old guarded codec has no corresponding Root branch. A meaningful global size comparison needs a new compiler and an implementation using one common fully specified public contract, rather than comparison of unmatched source lengths.
3. **Improve global certificate overhead.** The local three-node gadget is optimal in its stated arithmetic fragment. A proof DAG might share zero-frame reflexivities or earlier lifts across several root steps. Bounds on total nodes and serialized bytes for such a compiler remain separate questions.
4. **Settle successor rigidity.** The newest raw-clausal report proves that the restricted rule $f(0) = 0,\ f(Sz) = S(f(z)) \Rightarrow f(t) = t$ implies contextual induction for every zero equation $q(n) = 0$. Whether it recovers actual equality reflection and all equation induction in the scalar language is still open in the audited material. Predecessor and pairing cannot be imported before the missing bootstrap is supplied.
5. **Find a substantive arithmetic deletion.** The numerical theory already has strong compression. A more significant advance would remove an arithmetic obligation, a term constructor, or an entire logical operation while preserving both interpretation composites and reducing a complete matched checker. No such further deletion is claimed here.

Inherited logical and arithmetic reference proofs

The following five sections reproduce the reference development used by the audited raw-clausal report. Their original sources and the ProveIt licence are included in the archive. The named reference calculi in these sections are not extra primitive rules of $\mathcal{C}_2$. They make the inherited logic, predecessor, equality reflection, pairing, and both arithmetic composites reviewable inside this one document. Their mathematical content is credited to the source project, not presented as a new result of this report.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A,B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual

equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [8] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \end{aligned} \tag{10}$$

$$B_{Sm}(0) = C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \tag{11}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [9]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [10]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \leq b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F The complete recommended checker

The following is the complete standalone verifier, including the public input contract, type and scope validation, opening, inferences, JSON input, and acceptance or rejection. Its module docstring is included here for readability; the reported size uses the stated docstring-stripped normalization. The compiler and tests are separate untrusted tools and are not dependencies of this verifier.

```
"""Raw clausal iPRA with only zero and successor evaluation.

The baseline root equality case is derived through a zero-iteration frame.

Terms: 0, negative free levels, positive bound indices, [0,t] (S),
[n,a,h] (I). Formulas: [s,t] (equality), [B] (D).
Contexts contain (antecedent, one-bound-variable body) clauses.
Input: {'goal': a closed formula, 'proof': a finite proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def successor(a):
    return [0, a]


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) in (2, 3)
            and (len(a) == 3 or a[0] == 0)
            and all(term(t, scope, depth + (len(a) == 3 and i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    return [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
            for i, t in enumerate(a)]


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 2:
        a = proof[1]
```

```
        if not term(a, scope):
            fail()
        return [a, a]
    if tag == 'v' and len(proof) == 3:
        s, t = derive(proof[1], context, scope)
        b, r = derive(proof[2], context, scope)
        c, a, h = b
        if c != 0:
            if len(c) != 2:
                fail()
            a, t = [c[1], a, h], opening(h, t)
        if a != s:
            fail()
        return [r, t]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([0, 0], opening(body, eta))
            target = opening(body, successor(fresh))
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README and research index, commit `831081033253cca2115196cf86bb99c77ed2aeae`. https://github.com/VladimirReshetnikov/ProveIt/tree/831081033253cca2115196cf86bb99c77ed2aeae/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] *Five rules for iPRA with ordinary substitution*, `docs/incoming/ipra_raw_clausal_2026-11-11.zip`, Git blob `9c59217d9ea27fbd7542fcc47f90632d0f6fab64`, in the pinned project. Source of the raw clause calculus, the exact control checker, and its interpretation proofs.

[3] *Equality from iteration*, `docs/incoming/ipra_equality_from_iteration_2026-11-11.zip`, Git blob `3820db51b838a53e7019297829b44dc53d9c9da7`, in the pinned project. Source of the identity-frame deletion and zero-normal single-evaluation presentation.

[4] *Clausal canonical presentation*, `docs/incoming/ipra_clausal_canonical.zip`, Git blob `d5370834ca223e785b960e60efe8452c4283137a`, in the pinned project. In particular, `paper/five_rules.tex` gives clause use, the omitted closure binder, and both composites.

[5] ProveIt contributors, *Computational equality*, `Research/computational_equality/` and `Lean/ConciseIPRA/NativeEval.lean`, pinned project. The Lean module proves `eprf_iff_nprf` for the three-case Eval calculus. This report does not claim that the new clause checker has itself been formalized in Lean.

[6] ProveIt contributors, *One computation axiom for iPRA*, `Research/zero_normal/`, and the modules `Lean/ConciseIPRA/ZeroNormalization.lean` and `Lean/ConciseIPRA/NativeZeroNormal.lean`, pinned project. Normalization coherence and equivalence of the zero-normal kernel.

[7] ProveIt contributors, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, pinned project; reproduced in this report.

[8] ProveIt contributors, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, pinned project; reproduced in this report.

[9] ProveIt contributors, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, pinned project; reproduced in this report.

[10] ProveIt contributors, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, pinned project; reproduced in this report.

[11] ProveIt contributors, `Lean/ConciseIPRA/NativeBiInterpretation.lean` and `Lean/ConciseIPRA/BiInterpretation.lean`, pinned project. The existing formal native and scalar bi-interpretations with iPRA.

[12] ProveIt contributors, *Canonical recursors: compact companion*, `Research/canonical_recursors/compact/README.txt`, `ARGUMENT.txt`, and its `code/checker_compact_zero.py` and `code/checker_strict.py`, pinned project. The 516-byte compact auxiliary protocol and its 1376-byte strict guarded public interface; independently remeasured here.